Experiment # 06

Implementation of File I/O Redirection System Calls in Linux-I

Objective

In this lab we will learn about system call techniques in UNIX.

Software Tools

- Ubuntu 16.04
- GCC Complier

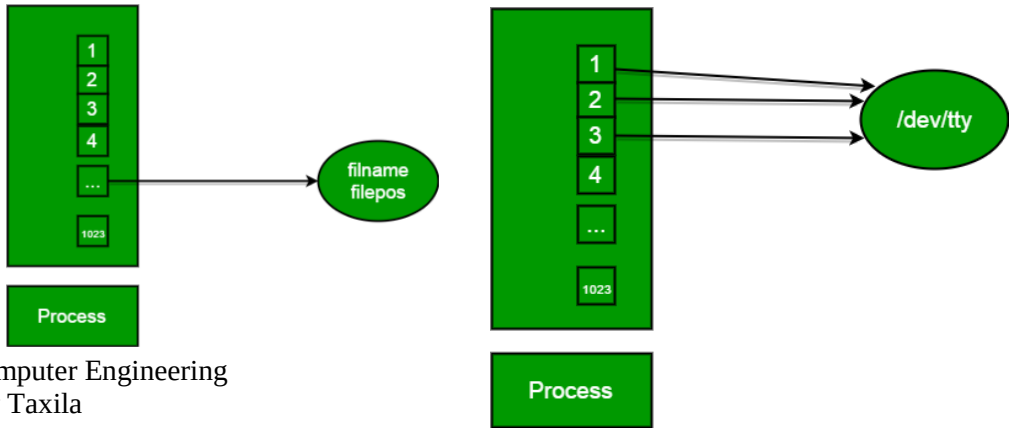
Theory

System Call:

System calls are commands that are executed by the operating system. "System calls are the only way to access kernel facilities such as file system, multitasking mechanisms and the inter-process communication primitives. File Structure related System Calls:

- create()
- open()
- read()
- write()
- close()

File Descriptors: A file descriptor uniquely identifies an open file of the process. Some file descriptor values are reserved for system uses. File descriptor is integer that uniquely identifies an open file of the process.
File Descriptor table: File descriptor table is the collection of integer array indices that are file descriptors in which elements are pointers to file table entries. One unique file descriptors table is provided in operating system for each process.
File Table Entry: File table entries is a structure In-memory surrogate for an open file, which is created when process request to opens file and these entries maintains file position.



Standard File Descriptors: When any process starts, then that process file descriptors table's fd(file descriptor) 0, 1, 2 open automatically, (By default) each of these 3 fd references file table entry for a file named **/dev/tty**
/dev/tty: In-memory surrogate for the terminal
Terminal: Combination keyboard/video screen
Read from stdin => read from fd 0 : Whenever we write any character from keyboard, it read from stdin through fd 0 and save to file named /dev/tty.
Write to stdout => write to fd 1 : Whenever we see any output to the video screen, it's from the file named /dev/tty and written to stdout in screen through fd 1.
Write to stderr => write to fd 2 : We see any error to the video screen, it is also from that file write to stderr in screen through fd 2.

FD	PURPOSE
0	Standard input
1	Standard output
2	Standard error
>3	User created file descriptors using the syscalls mentioned below

PERMISSIONS/MODE

Every file in unix has permissions, that is, which all users can read(r)/write(w)/execute(x) the file. These are often represented using octal numbers: In 0abc, a corresponds to the permissions of the owner, b corresponds to the permissions of the group, c corresponds to the permissions of others.

NUMBER	REF
0	---
1	--x
2	-w-
3	-wx
4	r--
5	r-x
6	rw-
7	rwx

For example, if the permissions of a file is 0600, then only the owner can read and write, but not execute the file. Other users cannot read, write or execute the file.

SYSCALLS

System call are those that the services of the operating system to the user programs via Application Program Interface(API). The different I/O syscalls used here, include:

1. Creat: Used to Create a new empty file.

Syntax in C language:

int creat(char *filename, mode_t mode)

Parameter:

- **filename :** name of the file which you want to create
- **mode :** indicates permissions of new file.

Returns:

- return first unused file descriptor (generally 3 when first create use in process because 0, 1, 2 fd are reserved)
- return -1 when error

How it work in OS

- Create new empty file on disk
- Create file table entry
- Set first unused file descriptor to point to file table entry
- Return file descriptor used, -1 upon failure

Purpose

Creates an empty file at path filename with permissions mode and returns the file descriptor that points to that file in the file table entry. In case of any error, it returns -1.

Sample usage

```
int new_file = creat("new_file.txt",0600);
```

Here, a new file with file name "new_file.txt" is created with the permissions 0600. The variable new_file contains the file descriptor

2. Open: Used to Open the file for reading, writing or both.

Syntax in C language

#include<sys/types.h>
#include<sys/stat.h>
#include <fcntl.h>
int open (const char* Path, int flags [, int mode]);

Parameters

- **Path:** path to file which you want to use
    - use absolute path begin with "/", when you are not work in same directory of file.
    - Use relative path which is only file name with extension, when you are

work in same directory of file.

- **flags :** How you like to use
- **O_RDONLY**: read only, **O_WRONLY**: write only, **O_RDWR**: read and write, **O_CREAT**: create file if it doesn't exist, **O_EXCL**: prevent creation if it already exists

How it works in OS

- Find the existing file on disk
- Create file table entry
- Set first unused file descriptor to point to file table entry
- Return file descriptor used, -1 upon failure

Purpose

Open the file at path Path with the flags flags and permissions mode and returns the file descriptor that points to that file in the file table entry. In case of any error, it returns -1.

The different flags available include

FLAG	PURPOSE
O_RDONLY	read only
O_WRONLY	write only
O_RDWR	read and write
O_CREAT	create file if it doesn't exist
O_EXCL	prevent creation if it already exists
__O_LARGEFILE	used for opening large files(>1GB)
O_TRUNC	Truncate the contents of the file, if the file already exists

Sample usage

1) Open a file for writing

```
int dest_file = open("./output.txt", O_CREAT | O_RDWR |_O_LARGEFILE | O_TRUNC,
0600);
```

2) Open a file for reading

```
int source_file = open("./input.txt", O_RDONLY |_O_LARGEFILE);
```

```
// C program to illustrate
// open system call
#include<stdio.h>
#include<fcntl.h>
#include<errno.h>
extern int errno;
int main()
{
   // if file does not have in directory
   // then file foo.txt is created.
   int fd = open("foo.txt", O_RDONLY | O_CREAT);

   printf("fd = %d/n", fd);

   if (fd ==-1)
   {
      // print which type of error have in a code
      printf("Error Number % d\n", errno);

      // print program detail "Success or failure"
      perror("Program");
   }
   return 0;
}
```

Output: fd = 3

3. Close: Tells the operating system you are done with a file descriptor and Close the file which pointed by fd.

Syntax in C language

#include <fcntl.h>
int close(int fd);

Parameter:

- **fd :**file descriptor

Return:

- **0** on success.
- **-1** on error.

How it works in the OS

- Destroy file table entry referenced by element fd of file descriptor table

– As long as no other process is pointing to it!
- Set element fd of file descriptor table to **NULL**

Purpose

Close the file. It returns 0 once the file is closed successfully and -1 in case of any error.

Sample usage:

```
close(dest_file);
```

```
// C program to illustrate close system Call
#include<stdio.h>
#include <fcntl.h>
#include<unistd.h>
int main()
{
    int fd1 = open("foo.txt", O_RDONLY);
    if (fd1 < 0)
    {
        perror("c1");
        exit(1);
    }
    printf("opened the fd = % d\n", fd1);

    // Using close system Call
    if (close(fd1) < 0)
    {
        perror("c1");
        exit(1);
    }
    printf("closed the fd.\n");
}
```

Output:

opened the fd = 3

closed the fd.

```
// C program to illustrate close system Call
#include<stdio.h>
#include<fcntl.h>
int main()
{
   // assume that foo.txt is already created
```

```
   int fd1 = open("foo.txt", O_RDONLY, 0);
   close(fd1);

   // assume that baz.tzt is already created
   int fd2 = open("baz.txt", O_RDONLY, 0);

   printf("fd2 = % d\n", fd2);
   exit(0);
}
```

Output:
fd2 = 3

Here, In this code first open() returns **3** because when main process created, then fd **0, 1, 2** are already taken by **stdin**, **stdout** and **stderr**. So first unused file descriptor is **3** in file descriptor table. After that in close() system call is free it this **3** file descriptor and then after set **3** file descriptor as **null**. So when we called second open(), then first unused fd is also **3**. So, output of this program is **3**.

4. Read: From the file indicated by the file descriptor fd, the read() function reads cnt bytes of input into the memory area indicated by buf. A successful read() updates the access time for the file.

Syntax in C language
#include<fcntl.h>
size_t read (int fd, void* buf, size_t cnt);

Parameters:

- **fd:** file descriptor
- **buf:** buffer to read data from
- **cnt:** length of buffer

Returns: How many bytes were actually read

- return Number of bytes read on success
- return 0 on reaching end of file

- return -1 on error
- return -1 on signal interrupt

Important points

- **buf** needs to point to a valid memory location with length not smaller than the specified size because of overflow.
- **fd** should be a valid file descriptor returned from open() to perform read operation because if fd is NULL then read should generate error.
- **cnt** is the requested number of bytes read, while the return value is the actual number of bytes read. Also, some times read system call should read less bytes than cnt.

Purpose

Read data upto count bytes from a file whose file descriptor is fd and store the results in buf pointer. It returns the number of bytes read. In case of any error, it returns -1.

Sample usage

```
char *c = (char *)malloc(1000);
int chk = read(source_file, c, 1000);
```

```
// C program to illustrate
// read system Call
#include<stdio.h>
#include <fcntl.h>
int main()
{
int fd, sz;
char *c = (char *) calloc(100, sizeof(char));

fd = open("foo.txt", O_RDONLY);
if (fd < 0) { perror("r1"); exit(1); }

sz = read(fd, c, 10);
printf("called read(% d, c, 10). returned that"
       " %d bytes were read.\n", fd, sz);
c[sz] = '\0';
printf("Those bytes are as follows: % s\n", c);
}
```

Output:

called read(3, c, 10). returned that 10 bytes were read.

Those bytes are as follows: 0 0 0 foo.

Suppose that foobar.txt consists of the 6 ASCII characters "foobar". Then what is the output of the following program?

```
// C program to illustrate
// read system Call
#include<stdio.h>
#include<unistd.h>
#include<fcntl.h>
```

```
#include<stdlib.h>

int main()
{
   char c;
   int fd1 = open("sample.txt", O_RDONLY, 0);
   int fd2 = open("sample.txt", O_RDONLY, 0);
   read(fd1, &c, 1);
   read(fd2, &c, 1);
   printf("c = %c\n", c);
   exit(0);
}
```

Output:
c = f

The descriptors ***fd1*** and ***fd2*** each have their own open file table entry, so each descriptor has its own file position for ***foobar.txt***. Thus, the read from ***fd2*** reads the first byte of ***foobar.txt***, and the output is **c = f**, not **c = o**.

5. Write: Writes cnt bytes from buf to the file or socket associated with fd. cnt should not be greater than INT_MAX (defined in the limits.h header file). If cnt is zero, write() simply returns 0 without attempting any other action.

Syntax in C language

#include <fcntl.h>
size_t write (int fd, void* buf, size_t cnt);

Parameters:

- **fd:** file descriptor
- **buf:** buffer to write data to
- **cnt:** length of buffer

Returns: How many bytes were actually written

- return Number of bytes written on success
- return 0 on reaching end of file
- return -1 on error
- return -1 on signal interrupt

Important points

- The file needs to be opened for write operations
- **buf** needs to be at least as long as specified by cnt because if buf size less than the cnt then buf will lead to the overflow condition.
- **cnt** is the requested number of bytes to write, while the return value is the actual number of bytes written. This happens when **fd** have a less number of bytes to write than cnt.
- If write() is interrupted by a signal, the effect is one of the following:
  -If write() has not written any data yet, it returns -1 and sets errno to EINTR.
  -If write() has successfully written some data, it returns the number of bytes it wrote before it was interrupted.

Purpose

Read data upto count bytes from the memory location pointed by buf to a file whose file descriptor is fd. It returns the number of bytes read. In case of any error, it returns -1.

Sample usage

```
char *c = (char *)malloc(1000);
int chk=write(dest_file, c, 1000);
```

```
// C program to illustrate
// write system Call
#include<stdio.h>
#include <fcntl.h>
main()
{
int sz;

int fd = open("foo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd < 0)
{
   perror("r1");
   exit(1);
}

sz = write(fd, "hello engineer\n", strlen("hello engineer\n"));
printf("called write(% d, \"hello engineer\\n\", %d)."
   " It returned %d\n", fd, strlen("hello engineer\n"), sz);

close(fd);
}
```

Output:
called write(3, "hello engineer\n", 12). it returned 11

Here, when you see in the file foo.txt after running the code, you get a "*hello engineer*". If foo.txt file already have some content in it then write system call overwrite the content and all previous content are ***deleted*** and only "*hello engineer*" content will have in the file.

Print "hello world" from the program without use any printf or cout function.

```
// C program to illustrate
// I/O system Calls
#include<stdio.h>
#include<string.h>
#include<unistd.h>
#include<fcntl.h>

int main (void)
{
   int fd[2];
   char buf1[12] = "hello world";
   char buf2[12];

   // assume foobar.txt is already created
   fd[0] = open("foobar.txt", O_RDWR);
   fd[1] = open("foobar.txt", O_RDWR);

   write(fd[0], buf1, strlen(buf1));
   write(1, buf2, read(fd[1], buf2, 12));

   close(fd[0]);
   close(fd[1]);

   return 0;
}
```

Output:
hello world

In this code, buf1 array's string ***"hello world"*** is first write in to stdin fd[0] then after that this string write into stdin to buf2 array. After that write into buf2 array to the stdout and print output "***hello world***".

Implementation of open(), read(), write() and close() System Calls

lseek():

It is the system call to change the location of read/write pointer of a given file descriptor.

Syntax:
off_t lseek(int file_descriptor, off_t offset, int whence);

- **file_descriptor**(The file whose current file offset you want to change).
- **offset**(The amount (positive or negative) the byte offset is to be changed. The sign indicates whether the offset is to be moved forward (positive) or backward (negative).
- **Whence**: One of the following symbols:
  - SEEK_SET: The start of the file
  - SEEK_CUR: The current file offset in the file
  - SEEK_END: The end of the file

Purpose
repositions the file offset of the file descriptor fd to offset bytes with respect to whence directive:
- SEEK_SET: set file offtset to offset bytes from the beginning of the file
- SEEK_CUR: set file offtset to offset bytes + current location
- SEEK_END: set file offtset to offset bytes + size of file

It returns the new location of the file offset of the file descriptor

Sample usage

1) Get the current location of the file descriptor

```
int current_location = lseek(source_file, 0, SEEK_CUR);
```

2) Move the file descriptor to the start of the file

```
lseek(source_file, 0, SEEK_SET);
```

3) Move the file descriptor to the end of the file

```
lseek(source_file, -1, SEEK_END);
```

4) Get the length of the file

```
off_t fileLength = lseek(source_file, 0, SEEK_END);
```

Example.c:

```
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
#include <sys/types.h>
#include <sys/stat.h>

int main()
{
int fd; char
buffer[80];
static char message[] = "Hello, world";

fd = open("created_file.txt",O_RDWR|O_CREAT);
printf("fd = %d \n", fd);

if (fd != -1)
{
printf("myfile opened for read/writeaccess\n");
write(fd, message, sizeof(message));

lseek(fd, 0, 0); /* go back to the beginning of the file */

read(fd, buffer, sizeof(message));
printf(" %s was written to myfile \n",buffer);

close (fd);
}

}
```

Lab Task:

1. **Execute all the examples given in the manual and analyze the output.**
2. **Open a new file for reading and writing. Read the predefined number from the text file and calculate its factorial. Now write the output number to the same file as: Factorial is: output_number.**

Lab Performance Evaluation:
Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.

Lab Report Evaluation:
Submit the Lab Report by writing all the examples and tasks which must include the following:

1. Brief description about how its work (3-5 lines)
2. The code
3. The results (Screenshot)

Important note: Lab Report must be according to the Lab Report Format available on Google Classroom and must be submitted on time. Two similar reports will get zero marks. So, don't try to copy your fellow reports. Lab report must be submitted in group of three maximum.